

AGENCYOS

API & Integration Specification

Document 22 of the AgencyOS Master Documentation Set

WHATSAPP • AI PROVIDERS • EMAIL • PAYMENT-RELATED INTEGRATIONS • WEBHOOKS •
EXTERNAL SERVICES

Purpose: Define the integration layer through which AgencyOS communicates with WhatsApp, AI model providers, email systems, payment services and other external platforms while preserving authentication, tenant isolation, idempotency, observability, retries, rate limits, auditability and failure recovery.

1. Integration Principles

- All external integrations are behind explicit adapters/connectors rather than scattered direct API calls.
- External systems are not treated as trusted sources merely because they authenticated successfully.
- Every integration is tenant-aware where applicable.
- Credentials are secret-managed and never exposed to agents unnecessarily.
- External events are validated, deduplicated and processed idempotently.
- Failures are observable and recoverable.
- External API changes are isolated behind provider adapters.
- Sensitive operations require authorization and policy checks.
- Internal domain state remains authoritative for AgencyOS business decisions unless an integration is explicitly designated as the source of truth.

2. Integration Architecture

AGENCYOS DOMAIN/API → INTEGRATION SERVICE → PROVIDER ADAPTER → EXTERNAL API.

INBOUND: EXTERNAL WEBHOOK → AUTHENTICATE/VERIFY → NORMALIZE → IDEMPOTENCY CHECK → DOMAIN EVENT → WORKFLOW.

OUTBOUND: DOMAIN EVENT/ACTION → POLICY CHECK → PROVIDER ADAPTER → API CALL → RESPONSE NORMALIZATION → AUDIT/OBSERVABILITY.

3. Integration Registry

- Integration ID.
- Provider.
- Integration type.
- Organization scope.
- Environment.
- Status.
- Credential reference.
- Capabilities.
- API/version metadata.
- Webhook configuration.
- Rate-limit configuration.
- Health status.
- Last successful call.
- Last failure.
- Created/updated timestamps.

4. Integration Lifecycle

UNCONFIGURED → CONFIGURED → CONNECTING → ACTIVE → DEGRADED → DISABLED → REVOKED.

- Credential rotation can occur without changing the integration identity.
- Disabled integrations reject new calls according to policy.

- Revoked credentials require reauthorization.

5. Provider Abstraction

- Expose canonical AgencyOS operations.
- Map provider-specific request/response formats inside adapters.
- Normalize errors.
- Normalize rate limits.
- Normalize usage/cost metadata where available.
- Provider-specific features remain optional capabilities.
- Do not let provider-specific payload shapes leak throughout the domain model.

6. API Standards

- Use versioned APIs.
- Use JSON/schema validation for structured requests.
- Use consistent error envelopes.
- Use request IDs/correlation IDs.
- Use idempotency keys for retry-sensitive operations.
- Use pagination for collections.
- Use filtering/sorting rules consistently.
- Use explicit timestamps/time zones.
- Never expose internal stack traces or secrets.

7. Authentication & Authorization

- Authenticate internal and external API calls.
- Authorize every protected operation.
- Use scoped API tokens/service identities.
- Apply organization/project context.
- High-risk integrations require additional policy checks.
- Credential possession is not equivalent to permission to perform a business action.

8. Secrets & Credentials

- Store API keys, OAuth client secrets, webhook secrets, payment credentials and provider tokens in secure secret management.
- Store only references/metadata in ordinary database records.
- Support rotation/revocation.
- Separate development, staging and production credentials.
- Mask secrets in logs.
- Do not pass raw credentials into AI prompts or agent-visible context.

9. WhatsApp Integration

- Support lead/client/internal communication workflows according to the selected WhatsApp provider/API.
- Map contacts to AgencyOS Contact/Client/Lead records.
- Map client groups/conversations to Project context where supported.

- Normalize inbound messages.
- Send approved outbound messages.
- Track delivery/read/status events where provider supports them.
- Handle text, media and voice-note metadata according to supported provider capabilities.
- Respect opt-in, template and messaging policies applicable to the provider.
- Do not treat a WhatsApp message as formal approval/payment verification unless the configured workflow explicitly supports and records that approval.

10. WhatsApp Inbound Flow

PROVIDER WEBHOOK → VERIFY SIGNATURE/AUTH → DEDUPE → IDENTIFY
CONTACT/PROJECT → NORMALIZE MESSAGE → STORE MESSAGE → CLASSIFY → ROUTE
TO AGENT/HUMAN → ACTION → RESPONSE → AUDIT.

11. WhatsApp Outbound Flow

TRIGGER → COMMUNICATION POLICY → TEMPLATE/CONTENT CHECK → RECIPIENT
CHECK → PROVIDER API → MESSAGE ID → DELIVERY TRACKING → AUDIT.

- Sensitive or high-risk messages can require Admin/agent approval according to policy.

12. WhatsApp Group Management

- Create/link project group where provider capabilities allow.
- Store provider group ID.
- Track members.
- Track agency/client roles where supported.
- Record group creation event.
- Record membership changes.
- Do not assume provider-side group state is always current; reconcile where possible.

13. WhatsApp Failure Handling

Rate limit → retry with backoff.

Temporary provider outage → retry/fallback channel.

Invalid recipient → mark failed and surface action.

Template rejection → route to approved template/content workflow.

Duplicate webhook → ignore after idempotency check.

Permanent failure → create communication exception.

14. AI Provider Integration

- Support multiple model providers such as OpenAI/ChatGPT, Claude, Gemini, Grok, DeepSeek and additional providers through a common abstraction.
- Provider registry stores capabilities, model names/versions, availability and configuration metadata.
- Routing policy chooses provider/model by agent/task.
- Fallback policy handles provider outage, quota, latency or policy constraints.

- Usage/cost metadata is captured where provider APIs support it.

15. AI Request Pipeline

AGENT TASK → CONTEXT AUTHORIZATION → MODEL ROUTING → PROVIDER ADAPTER → REQUEST → RESPONSE VALIDATION → TOOL/POLICY CHECK → RESULT → USAGE/AUDIT.

- Provider response is treated as untrusted content and cannot directly grant permissions.

16. AI Provider Security

- Sensitive context is minimized.
- Provider credentials remain server-side.
- Provider/model permissions are separate from agent permissions.
- Fallback providers inherit the same organization/data policy.
- Provider selection is auditable.
- Provider failure must not cause an unsafe automatic escalation of privileges.

17. AI Provider Health & Routing

- Track availability.
- Track latency.
- Track error rate.
- Track quota/limits where available.
- Track cost/usage.
- Track task quality signals where configured.
- Routing policy can choose preferred model and fallback.
- Admin can disable a provider/model.

18. Email Integration

- Support transactional email.
- Support project/client notifications.
- Support invoice/payment notifications.
- Support approval requests.
- Support password/account notifications.
- Support Customer Success/renewal communication.
- Use templates and variables.
- Track message ID/status where provider supports it.
- Separate transactional and promotional communication policies.

19. Email Inbound

- Receive replies where supported.
- Validate provider/webhook events.
- Map sender to contact/client/project.
- Store normalized message metadata.
- Route to communication/CRM workflow.

- Do not let arbitrary email text alter scope, payment or permissions automatically.

20. Payment-Related Integrations

- Support payment gateway/bank/UPI-related integrations through provider adapters where available.
- Provider configuration is organization-specific.
- Store external transaction/reference IDs.
- Receive payment webhooks.
- Validate webhook authenticity.
- Deduplicate transactions.
- Match to invoice/milestone.
- Create Payment/Payment Submission records.
- Require financial verification rules before marking payment verified.

21. Payment Webhook Flow

PAYMENT PROVIDER → WEBHOOK → SIGNATURE VERIFY → IDEMPOTENCY →
TRANSACTION NORMALIZATION → INVOICE MATCH → PAYMENT STATE UPDATE →
RECONCILIATION → RECEIPT/NOTIFICATION → AUDIT.

- Never mark payment as verified from an unverified webhook.

22. Payment Integration Failure Handling

Duplicate event → ignore after idempotency.

Unknown invoice → exception queue.

Amount mismatch → exception/review.

Signature failure → reject and log security event.

Provider outage → show pending state; do not fabricate success.

Delayed confirmation → retry/poll where supported.

23. External Service Categories

- File/object storage.
- Analytics/observability.
- Monitoring/alerting.
- Version-control/repository services.
- CI/CD/build services.
- Deployment/cloud infrastructure.
- Identity/OAuth providers.
- Search/indexing.
- Maps/location services if required.
- Calendar/scheduling services if later added.
- Other business APIs.
- Each service uses a dedicated adapter and explicit capability list.

24. Webhook Framework

- Every webhook has provider, endpoint, event type, signature method, secret reference and active status.
- Verify authenticity before processing.
- Store provider event ID.
- Use idempotency.
- Record received timestamp.
- Normalize payload.
- Store raw payload only when policy permits and sensitive data is protected.
- Create internal domain event.
- Process asynchronously when appropriate.
- Record processing result.

25. Webhook Security

- Verify signatures.
- Reject malformed payloads.
- Reject replayed/duplicate events.
- Use timestamp tolerance where supported.
- Rate limit webhook endpoints.
- Do not expose detailed failure information to attackers.
- Separate webhook secrets by provider/environment.
- Audit verification failures.

26. Retry Policy

- Retry only transient failures.
- Use exponential backoff with jitter.
- Cap retry count.
- Respect provider Retry-After instructions where available.
- Do not retry invalid credentials indefinitely.
- Do not retry permanent validation errors.
- Persist retry state for asynchronous jobs.

27. Idempotency

- Outbound payment/financial operations use idempotency where provider supports it.
- Webhook events use provider event IDs or equivalent keys.
- Agent retries do not create duplicate quotes, invoices, payments, messages or projects.
- Idempotency records have retention appropriate to the operation.

28. Rate Limits

- Store provider-specific limits where known.
- Implement per-provider and per-organization throttling.
- Protect internal services from runaway agent loops.
- Prioritize critical workflows according to policy.

- Expose rate-limit state to monitoring/routing systems.

29. Timeout & Circuit Breaker

- External calls have explicit timeouts.
- Repeated provider failures can open a circuit.
- Fallback provider/channel can be used when policy allows.
- Recovery closes the circuit after successful health checks.
- Do not leave agents waiting indefinitely on external APIs.

30. Integration Observability

- Request ID.
- Correlation ID.
- Provider.
- Endpoint/operation.
- Organization/project context where safe.
- Latency.
- Status.
- Retry count.
- Error class.
- External request/event ID.
- Usage/cost metadata where available.
- Never log raw secrets or unnecessary sensitive payloads.

31. Integration Audit Events

- Integration configured.
- Credential added/rotated/revoked.
- Provider enabled/disabled.
- Webhook received/verified/rejected.
- Payment event processed.
- AI provider/model selected.
- External message sent.
- External message failed.
- Deployment provider called.
- Integration override performed.

32. External API Error Model

- AUTHENTICATION_ERROR.
- AUTHORIZATION_ERROR.
- VALIDATION_ERROR.
- RATE_LIMITED.
- TIMEOUT.
- PROVIDER_UNAVAILABLE.
- NOT_FOUND.

- CONFLICT/DUPLICATE.
- PROVIDER_ERROR.
- NETWORK_ERROR.
- POLICY_BLOCKED.
- UNKNOWN_ERROR.
- Map provider-specific errors into canonical internal categories.

33. Integration State vs Domain State

- External provider state is stored separately from AgencyOS domain state.
- Example: message delivery status can be external state; client communication task remains AgencyOS state.
- Payment provider status is external evidence; Finance determines the authoritative verified payment state.
- AI provider availability affects routing, not agent authorization.

34. Integration Testing

- Mock provider contract tests.
- Provider sandbox tests.
- Webhook signature tests.
- Duplicate event tests.
- Retry tests.
- Timeout tests.
- Rate-limit tests.
- Invalid credential tests.
- Provider outage tests.
- Fallback tests.
- Tenant-isolation tests.
- End-to-end tests for critical integrations.

35. Contract Testing

- Maintain provider adapter contracts.
- Validate request schemas.
- Validate response normalization.
- Detect provider API changes.
- Pin/test API versions where applicable.
- Do not allow silent provider response changes to corrupt domain state.

36. Integration Configuration in Admin Panel

- Provider selection.
- API credentials through secure secret references.
- Model/provider activation.
- WhatsApp configuration.
- Email provider configuration.
- Payment integration configuration.
- Webhook configuration.

- Rate-limit settings.
- Retry policies.
- Fallback policies.
- Health status.
- Test connection.
- Enable/disable integration.
- All sensitive changes are audited.

37. Agent Access to Integrations

- Agents access integrations through approved tools/services.
- Sales Agent can use approved CRM/communication integrations.
- PM Agent can use project communication and notification tools.
- Finance Agent can use payment/invoice services according to permission.
- Developer Agent can use repository/CI services for assigned project.
- QA Agent can use test/build services.
- Customer Success can use client communication/support services.
- Agent cannot directly retrieve arbitrary provider credentials.

38. External Service Permissions

- Each integration capability has required permission.
- Example: SEND_MESSAGE, READ_MESSAGE, CREATE_GROUP, VERIFY_PAYMENT, CREATE_INVOICE, RUN_BUILD, DEPLOY_RELEASE, QUERY_AI_MODEL.
- Permission checks happen before provider call.
- High-risk capabilities require policy checks.

39. Data Mapping

- Map external IDs to internal stable IDs.
- Store provider and provider_object_id.
- Keep mapping unique within provider/context.
- Normalize dates/time zones.
- Normalize phone numbers/emails where appropriate.
- Do not duplicate authoritative business records unnecessarily.

40. Synchronization Strategy

- Push events when provider supports webhooks.
- Poll only when necessary.
- Use periodic reconciliation for critical external state.
- Record last-sync timestamp.
- Track sync errors.
- Allow Admin to trigger resync.
- Resync must be idempotent.

41. External Data Conflicts

- Identify source-of-truth per field.
- Do not blindly overwrite internal approved state.
- Create conflict record where required.
- Route financial/security/approval conflicts to authorized review.
- Preserve both external and internal evidence.

42. Integration Availability & Degradation

- If WhatsApp is down, queue or use configured alternate channel.
- If an AI provider is down, route to approved fallback.
- If email is down, retain notification task and retry.
- If payment provider is unavailable, preserve pending financial state.
- If deployment provider is unavailable, block/hold deployment rather than claiming success.

43. Integration Lifecycle Management

Discover → Configure → Test → Activate → Monitor → Rotate/Update → Disable/Revoke.

- Provider API changes require controlled adapter updates.
- Decommissioning requires impact assessment and data migration/retention decision.

44. API Versioning & Backward Compatibility

- Internal APIs are versioned where breaking changes are possible.
- External provider versions are isolated in adapters.
- Use compatibility periods for critical integrations.
- Deprecation is documented.
- Do not break existing webhooks without migration support.

45. API Documentation

- Document endpoints.
- Authentication.
- Request/response schemas.
- Errors.
- Idempotency.
- Rate limits.
- Webhook events.
- Examples.
- Version.
- Permissions.
- Operational notes.

46. Integration Secrets Rotation

- Create new credential.
- Validate new credential.
- Switch active reference.

- Verify integration.
- Revoke old credential.
- Audit rotation.
- Never expose credential values during the process.

47. Integration Incident Workflow

DETECT → CLASSIFY → CONTAIN → RETRY/FALLBACK → ESCALATE → RECOVER → RECONCILE → CLOSE.

- Critical provider incidents can pause related workflows.
- Financial/security incidents follow their specialized incident policies.

48. Integration Acceptance Criteria

- All external services use adapters/connectors.
- Credentials are securely managed.
- WhatsApp workflows are supported.
- AI providers are multi-model and policy-routed.
- Email workflows are supported.
- Payment integrations support verified transaction handling.
- Webhooks are authenticated and idempotent.
- Retries/backoff/timeouts exist.
- Rate limits are respected.
- Circuit breaking/fallback exists where appropriate.
- Tenant isolation is enforced.
- Integration actions are auditable.
- Provider failures do not create false success states.
- Admin can configure/disable integrations.
- Agent permissions are enforced before external calls.
- External IDs map safely to internal records.
- Critical integrations have contract and failure tests.

49. Integration Matrix

Integration	Primary AgencyOS Use	Inbound	Outbound	Critical Controls
WhatsApp	Lead/client/internal communication	Messages/status webhooks	Messages/groups	Opt-in/policy, auth, dedupe, audit
AI Providers	Agent intelligence	Provider responses	Model requests	Routing, permissions, secrets, cost/usage
Email	Notifications/approvals/support	Replies/events	Transactional email	Auth, templates, dedupe, privacy
Payments	Payment collection/verification	Payment webhooks	Payment requests/links where supported	Signature, idempotency, reconciliation
Repositories/CI	Development/build/test	Build/status events	Commits/build/test calls	Scoped credentials, project isolation
Deployment/Cloud	Production delivery	Deployment events	Deploy/rollback calls	Approval gates, secrets, audit
Storage	Files/artifacts	Upload/status events	Upload/download	Scoped access, signed URLs

Monitoring	Health/alerts	Alerts/metrics	Query/config where supported	Access control, incident workflow
------------	---------------	----------------	------------------------------	-----------------------------------

50. Final Integration Architecture

AGENCYOS DOMAIN → POLICY/AUTHORIZATION → INTEGRATION SERVICE → PROVIDER ADAPTER → EXTERNAL API

EXTERNAL WEBHOOK → VERIFY → DEDUPE → NORMALIZE → DOMAIN EVENT → WORKFLOW → AUDIT

Every external dependency should be replaceable without rewriting the AgencyOS domain model.
External failure must produce a controlled pending/error state—not a fabricated success state.

Key outcome: AgencyOS becomes integration-ready without becoming provider-dependent. WhatsApp, AI platforms, email, payment systems and future services can be added, replaced, disabled or routed through common controls while the internal business logic remains stable.